

Formula Newton cu diferențe divizate

Prezentăm formula Newton cu diferențe divizate pentru polinomul de interpolare, datorată matematicianului englez Isaac Newton.

Definiția 2

$$N_n(x) = [y_0] + [y_0, y_1] \cdot (x - x_0) + [y_0, y_1, y_2] \cdot (x - x_0) \cdot (x - x_1) + \cdots + [y_0, y_1, \dots, y_n] \cdot (x - x_0) \cdot (x - x_1) \cdot \dots \cdot (x - x_{n-1}),$$

$$\text{unde } [y_h, y_{h+1}, \dots, y_k] = \begin{cases} y_h, & \text{dacă } h = k \\ \frac{[y_{h+1}, \dots, y_k] - [y_h, \dots, y_{k-1}]}{x_k - x_h}, & 0 \leq h < k \leq n \end{cases} \text{ (diferențe divizate).}$$

Teorema 3

$N_n \equiv P$, adică N_n este o formulă pentru polinomul de interpolare.

Demonstrație

Prin inducție matematică.

Pentru a calcula eficient coeficienții polinomului Newton (cu diferențele divizate), facem notația:

$$a_{hk} = [y_h, y_{h+1}, \dots, y_k], 0 \leq h < k \leq n.$$

Din Definiția 2 obținem:

$$a_{hk} = \begin{cases} y_h, & \text{dacă } h = k \\ \frac{a_{h+1k} - a_{hk-1}}{x_k - x_h}, & \text{dacă } h < k, 0 \leq h < k \leq n. \end{cases}$$

Pornind de la formula de mai sus, observând că a_{hk} depinde de a_{h+1k} (elementul din de dedesubt) și de a_{hk-1} (elementul stânga), rezultă imediat un algoritm (din categoria programării dinamice) pentru calculul valorilor a_{hk} :

Algoritm pentru calculul valorilor a_{hk}

Pasul 1.

for $h = 0$ **to** n **do**

$$a_{hh} = y_h;$$

end for;

Pasul 2.

for $h = n - 1$ **down to** 0 **do**

for $k = h + 1$ **to** n **do**

$$a_{hk} = \frac{a_{h+1k} - a_{hk-1}}{x_k - x_h};$$

end for;

end for;

Folosind algoritmul în doi pași de mai sus, se obține o matrice triunghiulară superioară $\mathcal{A} = (a_{hk})_{h,k=\overline{0,n}}$ (având elemente numai pe diagonala principală și deasupra acesteia). Din această matrice ne interesează numai prima linie, care dă coeficienții polinomului Newton.

Calculul coeficienților polinomului Newton se face în $O(n^2)$. După ce coeficienții sunt determinați, calculul lui $N_n(\xi)$ (valoarea polinomului Newton într-un punct ξ) se poate face în complexitate $O(n)$:

Algoritm de calcul al valorii polinomului Newton cu diferențe divizate într-un punct ξ

$v = a_{00};$

$p = \xi - x_0;$

for $h = 1$ **to** n **do**

$v += a_{0h} * p;$

$p *= (\xi - x_h);$

end for;

Pentru calculul a m puncte avem complexitatea $O(n^2 + m \cdot n)$ (am inclus aici și calculul coeficienților). Este clar că formula Newton este mai eficientă d.p.d.v. al complexității calculelor atât decât formula Lagrange, cât și decât formula ce se obține aplicând Cramer.

În cazul în care valorile x_k sunt uniform distribuite ($x_k - x_{k-1} = x_1 - x_0, k = \overline{1,n}$), pentru polinomul de interpolare există formule Newton (mai simple) cu diferențe finite la stânga și, respectiv, la dreapta. Deși complexitatea calculelor ce se efectuează este mai mică în cazul

formulelor cu diferențe finite, complexitatea temporală a calculelor rămâne însă de același ordin ca și în cazul formulei cu diferențe divizate. Mai mult, formula Newton cu diferențe divizate funcționează într-un caz mai general (când valorile x_k nu sunt uniform distribuite).

Să observăm în final că, în cazul formulei Newton cu diferențe divizate, adăugarea unui punct de control nou $A_{n+1}(x_{n+1}, y_{n+1})$ cu abscisa x_{n+1} mai mare decât a celorlalte puncte de control se face prin adăugarea la sfârșitul matricei $\mathcal{A}$ a unei coloane noi, efectuând calcule aferente acestuia în complexitate $O(n)$, adică nu trebuie recalculată întreaga matrice $\mathcal{A}$ în complexitate $O(n^2)$. Practic, pentru ultima coloană, nou introdusă, se execută:

Algoritm de adăugare a unei coloane la sfârșitul matricei triunghiulare

$a_{n+1n+1} = y_{n+1};$

for $h = n$ **down to** 0 **do**

$$a_{hn+1} = \frac{a_{h+1n+1} - a_{hn}}{x_{n+1} - x_h},$$

end for;